FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder

A Main Project Report

Submitted to the APJ Abdul Kalam Technological University
in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering(AI)

by

ADITHYAN R S *(B23CS2109)*
ALONA BIJU *(B23CS2113)*
ALVIN BINU *(B23CS2114)*
DARSAN M *(B23CS2125)*

Department of Computer Science and Engineering
Mar Baselios College of Engineering and Technology
(Autonomous)
Mar Ivanios Vidya Nagar, Nalanchira
Thiruvananthapuram–695015

October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MAR BASELIOS COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
MAR IVANIOS VIDYA NAGAR, NALANCHIRA
THIRUVANANTHAPURAM–695015

CERTIFICATE

This is to certify that the report entitled **FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder** submitted by **Adithyan R S** (KTU Reg No: B23CS2109), **Alona Biju** (KTU Reg No: B23CS2113), **Alvin Binu** (KTU Reg No: B23CS2114) and **Darsan M** (KTU Reg No: B23CS2125) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering (AI) is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Poorna B R
(Project Guide)
[TODO: designation]
Department of CSE

Dr. Ann Alex
(Head of the Department)
Professor
Department of CSE

Place: Thiruvananthapuram Date:

Declaration

We hereby declare that the project report entitled **FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder**, submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI), is a record of original work carried out by us at the Department of Computer Science and Engineering, Mar Baselios College of Engineering and Technology (Autonomous), Thiruvananthapuram. The experimental results reported here were produced by the system described in this report. Wherever the work of others has been used, it has been duly acknowledged and cited. This report has not been submitted, in whole or in part, to any other University or Institute for the award of any degree or diploma.

Place: Thiruvananthapuram
Date:

Adithyan R S (B23CS2109)
Alona Biju (B23CS2113)
Alvin Binu (B23CS2114)
Darsan M (B23CS2125)

Acknowledgement

We express our deepest gratitude to our **Project Guide** from the Department of Computer Science and Engineering for continuous guidance, critical feedback and technical insight throughout this project. Their advice on experimental rigour and on stating results precisely shaped both the direction and the quality of this work.

We sincerely thank **Dr. Ann Alex**, Head of the Department of Computer Science and Engineering, for academic support, provision of resources and encouragement at every stage of the project.

We acknowledge the open-source communities behind **PyTorch**, **TransformerLens**, **SAELens**, **Hugging Face** and **Streamlit**, the author of the pretrained **gpt2-small-res-jb** sparse autoencoders, and the **Neuronpedia** project. We also thank the authors of ROME, whose openly released fact datasets, including CounterFact, formed the basis of the benchmarks, and the authors of the model-stitching work on which our translators build.

Finally, we are grateful to our families and friends for their patience and encouragement throughout this endeavour.

Adithyan R S
Alona Biju
Alvin Binu
Darsan M

Abstract

Language models sometimes rank a correct completion below competing words even though the answer is within reach. Fixing this by changing weights is permanent and degrades as edits accumulate; adding text to the prompt does not use what the model already computes. This project, **FeatureScalpel**, asks whether the fix can be **transient and weight-free**: during one forward pass the residual stream of GPT-2 Small is edited through the features of a pretrained **sparse autoencoder (SAE)**, and the edit disappears when the pass ends.

The report follows the work as a record of trial and error. Muting the target's own helpers corrected 1 of 22 test facts and muting the leading competitor 3 of 22; muting alone failed on the probe prompt and mute with boost reached rank 1. Safety filtering, a study of depth, a batched implementation (186 to 6 model calls per prompt and layer), pool refill and candidates from all prompt positions followed. A small network that chose the strengths was no better per prompt than one learned pair, but tuning one multiplier per feature by gradient descent was far more effective: on 300 held-out CounterFact prompts it brought the true answer to rank 1 on 271, against 161 for a sweep given the same range and 94 for the original settings, with lower side effects and 5 to 6.5 times less compute. An additive edit then forced random unrelated words to rank 1 on 20 of 20 prompts, so rank 1 is weak evidence about knowledge. A side study applied the tools to a new question, carrying an edit from GPT-2 Small to Medium: a linear translator keeps the receiver working, an exported edit lifts the target's median rank from 142 to 14 but reaches rank 1 on only 8% of records, what crosses is a push toward a word and not a fact, and a half-and-half mix of Medium's state raises Small's accuracy from 8.5% to 9.5%. Pass bars of the side study were fixed before the runs; the limits (one model family, one dataset, no comparison with ROME or in-context editing) are stated.

Table of Contents

Acknowledgement i

Abstract ii

List of Figures vi

List of Tables vii

1 Introduction 1

1.1 Background and Motivation 1
1.2 How the Project Grew 1
1.3 Problem Statement and Research Questions 2
1.4 Objectives, Scope and Contributions 3
1.5 Organisation of the Report 3

2 Literature Review 4

2.1 Knowledge Editing 4
2.2 Activation Steering and Sparse Autoencoders 4
2.3 Causal Localisation and Attribution 5
2.4 Gradient-Based Optimisation of Edits and Steering Vectors 5
2.5 Comparing and Stitching Representations 6
2.6 Research Gap 6

3 The Edit and How It Works 7

3.1 Setting and Notation 7
3.2 SAE Features and Delta Patching 7
3.3 Choosing Candidates: Causal Ranking 8
3.4 The Hybrid Mute and Boost Sweep 8
3.5 The Gradient-Descent Editor 9
3.6 The Additive Edit 9
3.7 Benchmark and Metrics 10

4 Evolution of the System: a Record of Trial and Error **12**
4.1 The Starting Point . . . 12
4.2 Single-Feature Ablation . . . 12
4.3 Several Competitors, Spread-Out Support . . . 13
4.4 Mute and Boost Together . . . 13
4.5 Where to Edit, and Making It Fast . . . 14
4.6 Making the Sweep Reliable . . . 14
4.7 Is the Layer, or the SAE, the Limit? . . . 15
4.8 Scaling Up: the CounterFact Benchmark . . . 15
4.9 Learning the Strengths . . . 16
4.10 Gradient Descent Enters . . . 17
4.11 Forcing Every Target to Rank 1: the Additive Edit . . . 17
4.12 What the Edit Does Beyond the Next Word . . . 19
4.13 Where the Main Line Stands . . . 19

5 Applied Study: Carrying an Edit to Another Model **20**
5.1 The Question and the Design . . . 20
5.2 The Channel Works . . . 21
5.3 Export: a Real but Weak Effect . . . 22
5.4 Searching for a Better Translator . . . 22
5.5 What Is Carried: a Fact or a Push? . . . 23
5.6 Import: Medium's State into Small . . . 24
5.7 What the Side Study Shows . . . 24

6 Implementation **26**
6.1 Software, Hardware and Application . . . 26
6.2 Correctness Checks and Engineering Lessons . . . 27
6.3 Reproducibility and Development Process . . . 27

7 Discussion, Conclusion and Future Work **28**
7.1 What the Work Shows . . . 28
7.2 Threats to Validity and What Is Not Shown . . . 28
7.3 Conclusion . . . 29
7.4 Future Work . . . 29

References **30**

Appendices **33**
Appendix A: Research Journal and Report Sections . . . 33
Appendix B: Pass Bars of the Side Study and Their Verdicts . . . 33

Appendix C: Reproducing the Results 34

List of Figures

1.1 The path of the project in twelve stages (J = research journal entries). Stages 1 to 11 are the main line (Chapters 3 and 4); stage 12 is an applied side study (Chapter 5). 2

3.1 The gradient-descent editor. Only the free numbers z_k are updated. 10
3.2 Why an additive edit was tried. A multiplier leaves a silent feature (f3, f5) at zero; an additive amount c switches it on. 10

4.1 The early experiments. (a) 22 suppressed facts: muting the target's own helpers corrected 1, muting the competitor's drivers 3. (b) The probe prompt: muting alone stayed at rank 8; mute and boost reached rank 1 and a fourth boost undid it. 13
4.2 Making the computation fast. (a) Model calls per prompt and layer. (b) Whole Hybrid run, same result, three repeats. (c) CPU against GPU on 3 prompts and 4 layers. 14
4.3 Final rank of the target on the 13 comparable prompts when candidates come from the last token only or from all prompt positions (log scale; rank 1 is the dashed line). 15
4.4 Headroom on the 1,450 cached prompts (one-shot edits on saved states, a stand-in for the real sweep): (a) share reaching rank 1 by method; (b) free multipliers by starting-rank band. 16
4.5 Learned strengths on the 300 test prompts (stand-in measure): (a) prompts reaching rank 1; (b) mean KL of the successes. 17
4.6 Prompts brought to rank 1 by starting rank of the true answer, 300 held-out prompts: the original sweep, the sweep with the same 0 to 3 range, and gradient descent (green). 18
4.7 Control with random unrelated targets: the additive edit reaches rank 1 on 20 of 20 prompts at a size above the residual stream's; multipliers on 1 of 20. . . 18

5.1 The two directions studied. A translator maps the residual stream of one model into the other's coordinates; no weight changes, and the SAE is used only where the edit is made. 21

5.2 Top-1 of every export variant on the same 88 test records. Blue: map fitted on text; green: map trained on edits; red: per-sentence tuning against medium's output (a ceiling, not a transfer). 23
5.3 Import by mixing small's own state with medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy: best at a 50/50 mix, below small alone under full replacement. 24

List of Tables

4.1 What the trial-and-error record kept and what it dropped. 19

5.1 Export of the edit through the ridge map (88 primary test records; dose chosen on dev). 22

A.1 Journal entries and where they appear in this report. 33

B.1 Pass bars (“gates”) and their verdicts. 34

Chapter 1

Introduction

1.1 Background and Motivation

Large language models sometimes complete a factual prompt with a generic word while the correct answer sits a few places lower in the output ranking. For "The location of Massachusetts Institute of Technology is in", GPT-2 Small [1] may rank " the" first and " Cambridge" eighth. We call such cases **suppressed factual recall**; the term describes the output ranking only and makes no claim about what the model "knows". The usual remedies change the model: fine-tuning, or closed-form weight edits such as ROME [2] and MEMIT [3], which are permanent and, applied repeatedly, first erase earlier edits gradually and then collapse the model [4]. Retrieval and in-context editing [32, 29] leave the weights alone but supply the fact as extra text.

Mechanistic interpretability offers a third route. A sparse autoencoder (SAE) [6, 7] re-expresses a layer's activations in a much wider, sparsely active basis whose directions, called *features*, often match single concepts. A feature is a direction in the residual stream, so it can be turned down (*muted*) or up (*boosted*) during one forward pass and discarded afterwards. If this works, a wrong answer can be repaired **transiently**, without touching any weight, and the repair can be read as a short list of named features. This project asks whether it works, how it should be built, and what it really does.

1.2 How the Project Grew

The project, **FeatureScalpel** (early working name: Causal Feature Steering), started in July 2026 with a modest prototype and grew by trial and error. Every step was recorded in a numbered research journal (37 entries by October 2026), and this report follows that record in order. Figure 1.1 shows the whole path.

Two things should be clear from the start. The **main line** (stages 1 to 11) is the project: a transient SAE edit of GPT-2 Small, how to choose the edit (by hand-built search, by a trained



Figure 1.1: The path of the project in twelve stages (J = research journal entries). Stages 1 to 11 are the main line (Chapters 3 and 4); stage 12 is an applied side study (Chapter 5).

network, or by gradient descent), and what the edit does besides the one number it is tuned for. The **cross-model transfer study** (stage 12) is not a later version of the system. It is an *applied side study*: once the edit, the controls and the benchmark existed, we used them on a new problem to see whether the interpretability and intervention work had fruits beyond the single model it was built for.

1.3 Problem Statement and Research Questions

Given a prompt whose true answer GPT-2 Small ranks below first, can a transient edit of SAE features during one forward pass bring the answer to the top, how should the edit be chosen, and what does reaching the top mean? Five research questions organise the work.

1. **RQ1 (what works).** Which way of choosing features and strengths brings a suppressed answer to rank 1, and which ideas fail?
2. **RQ2 (limits).** What limits such an edit: the depth of the intervention, the safety rule, or the supply of features it can use?
3. **RQ3 (choosing the edit).** Can a program choose the edit better than a hand-built search: a trained network, or gradient descent on the edit itself?
4. **RQ4 (meaning of success).** Is rank 1 evidence that a fact was recovered? What else

does the edit change: reworded prompts, neighbouring facts, unrelated prompts, generated text?

5. **RQ5 (application).** Can the change an edit makes inside one model be carried into another model at inference time, without weight change, and what exactly is carried?

1.4 Objectives, Scope and Contributions

The objectives are to build a transient edit that changes only the intended feature contribution; to document with numbers what each design decision bought and which ideas failed; to build a benchmark with held-out prompts and exact paired statistics; to measure side effects with controls; and to apply the tools to the transfer question with pass bars fixed before each run. The study uses GPT-2 Small with the SAE release `gpt2-small-res-jb` (24,576 features per layer) at layer 8 in all later studies, single-token facts from ROME's `known_1000` and CounterFact [2], and next-token measurements; the side study adds GPT-2 Medium (no SAE). Other models, comparison with ROME, MEMIT or in-context editing, and a full evaluation of generated text are outside the scope.

The contributions are: (1) a documented trial-and-error record from single-feature ablation to gradient descent (Chapter 4); (2) a transient edit with measured design choices (delta patching, causal ranking, a strict per-feature safety filter, pool refill, candidates from all prompt positions: 10 of 13 prompts at rank 1 against 4) and a batched implementation that cut model calls per prompt and layer from 186 to 6 with identical results; (3) an equal-budget result: on 300 held-out CounterFact prompts gradient descent reached rank 1 on 271, against 161 for a sweep given the same range and 94 for the original settings, with lower same-prompt side effects and 5 to 6.5 times less compute; (4) a control showing that rank 1 is a weak test (an additive edit forces random words to rank 1 on 20 of 20 prompts), which led to withdrawing an early "knowledge absent" classification; and (5) an applied side study with fixed pass bars: a linear translator between GPT-2 Small and Medium, Export and Import tests with controls, a documented search that locates the limit, and a first positive Import result.

1.5 Organisation of the Report

Chapter 2 reviews related work, including where gradient-based optimisation of edits comes from. Chapter 3 gives the mathematics of the final system, Chapter 4 the story of how it was reached, and Chapter 5 the cross-model side study. Chapter 6 describes the application and engineering, and Chapter 7 the findings, limits and future work. The appendices map journal entries to sections, list the side study's pass bars and give reproduction commands.

Chapter 2

Literature Review

This chapter reviews the research threads on which the project draws, places the project's use of gradient descent in that literature, and states the gap. Most sources were examined through their abstracts and summaries, not in full; the numbers quoted are the sources' own.

2.1 Knowledge Editing

ROME [2] locates a factual association in the mid-layer MLPs of GPT-style models and rewrites one weight matrix in closed form; MEMIT [3] extends this to thousands of edits. Both change the weights permanently. Gupta *et al.* [4] showed that sequential edits first erase earlier edits gradually and then collapse the model abruptly, and Hase *et al.* [28] found that where a fact is localised need not predict where it can be edited best. ROME also contributed **CounterFact**, which pairs each prompt with a true and a counterfactual answer and supplies reworded (paraphrase) and neighbouring (neighbourhood) prompts for testing generalisation and specificity, and the `known_1000` set of facts its model already predicts; ROME pre-filters facts in this way, and so do we. Methods that leave the weights alone add the fact as text: in-context editing [29] places the corrected fact and demonstrations in the prompt, and retrieval-augmented generation [32] conditions the model on retrieved documents. Both change the input, not the model's internal computation. EasyEdit [30] collects these methods in one framework and is the intended route for baselines we have not run.

2.2 Activation Steering and Sparse Autoencoders

Activation steering adds a direction to the residual stream at inference time: Activation Addition [9] uses the difference of two prompts' activations, contrastive activation addition [10] averages such differences, and representation engineering [11] reads and controls population-level directions. Because superposition [5] makes single neurons rarely correspond to single concepts, these directions carry unrelated content along. Sparse autoencoders [6, 7, 8] learn

a wide sparse basis whose features are often interpretable; open SAEs exist for GPT-2 Small [15] and Gemma [17], with browsable explanations [18]. Several studies edit SAE features directly: Chalnev *et al.* [26] target features to limit side effects of steering, Zhao *et al.* [27] edit a few features to control whether a model uses its memory or the context, Farrell *et al.* [25] unlearn knowledge by feature ablation, Marks *et al.* [31] use features as nodes of editable circuits, and Arad *et al.* [21] show that choosing features by their effect on the output improves steering two- to threefold, which is the idea behind our causal ranking. The evidence is mixed. AxBench [12] reports that simple baselines can beat SAEs for steering, a Google DeepMind update [22] reports negative SAE results on downstream tasks, Chaudhary and Geiger [23] find SAEs struggle to reach a neuron baseline on factual knowledge in GPT-2 Small, and Cui *et al.* [24] show that behaviour suppressed by clamping SAE features can be recovered through the part of the activation the SAE does not explain. None of these tests our exact setting, but together they are why we measure side effects and controls and assume nothing about a feature edit.

2.3 Causal Localisation and Attribution

Activation patching measures the causal effect of an internal component [19]. Because exhaustive patching is expensive, gradient-based approximations exist: attribution patching and AtP* [16] estimate component effects from one backward pass. We rank features by actual ablation, evaluated in batches (Section 3.3), and keep gradient-based ranking as an option for larger fact sets. The logit lens [34] reads an intermediate state through the output projection and is used in our diagnostics.

2.4 Gradient-Based Optimisation of Edits and Steering Vectors

The project reached gradient descent from its own data (Section 4.10). It is, however, an old tool in this area, and the following lines of work place our use of it.

- **ROME itself.** Before it changes any weight, ROME finds the vector to be written by gradient descent on the probability of the new answer, with a term that keeps the subject's meaning unchanged [2]. Our loss has the same two ingredients (reach the target, limit the change elsewhere); the difference is that we tune a handful of SAE multipliers per prompt and change no weight.
- **Soft prompts and prefix tuning.** Lester *et al.* [42] and Li and Liang [43] keep the model frozen and train continuous vectors by gradient descent, which shows that a frozen model can be steered through a small optimised input.

- **Optimised steering vectors.** Subramani *et al.* [44] find vectors by gradient descent that, added to the hidden states of GPT-2, make it produce a chosen sentence nearly perfectly; Dunefsky and Cohan [45] optimise a steering vector from one training example and study how far it generalises, including to harmful behaviour.
- **Adversarial examples.** Szegedy *et al.* [46] and Goodfellow *et al.* [47] showed that a small perturbation found by gradient descent can force a network to any chosen output, and Zhang *et al.* [48] showed that networks can fit random labels. Both are the right comparison for our additive edit (Section 4.11): if an optimiser is given enough freedom, reaching the chosen output proves little.
- **The optimiser.** All our tuning uses Adam [33].

As far as our limited search found, none of these tunes one multiplier per *SAE feature* for a single prompt with a side-effect term and compares it with a hand-built feature sweep on held-out prompts.

2.5 Comparing and Stitching Representations

Model stitching connects the lower layers of one network to the upper layers of another through a small learned layer; if the stitched network still works, the two representations are compatible [37, 38]. Representational similarity measures such as CKA [39] compare activations without stitching. For language models, Chen *et al.* [40] show that a simple affine map between the residual streams of two models transfers features at low cost: they move SAEs from a smaller to a larger model (cutting the larger SAE's training cost by about half) and recover probes and steering vectors. Our translator (Chapter 5) is a ridge regression [41] of this family and is not claimed as new.

2.6 Research Gap

Knowledge editing changes weights or the prompt; SAE steering works inside one model; stitching work transfers vectors learned in one model. To our knowledge, based on a limited search, the following were not found together: a joint mute-and-boost edit of causally ranked SAE features for a single fact with a per-feature safety filter and pool refill; a comparison on held-out CounterFact prompts, under one yardstick, between a hand-built sweep and per-feature gradient descent; a control showing that an additive edit forces unrelated words to rank 1; and a test of whether the change an edit makes in one model can be carried into another, with pass bars fixed beforehand. This report supplies these, including the negative results. It does not claim that the SAE route beats prompting, probes, difference-of-means steering, ROME or in-context editing; that comparison has not been made.

Chapter 3

The Edit and How It Works

This chapter gives the mathematics of the final system, as implemented in `src/hooks.py`, `src/editing.py`, `src/hybrid_runner.py` and `src/gradient_editing.py`. Chapter 4 then shows how each piece was reached. The transfer-specific method is in Chapter 5.

3.1 Setting and Notation

GPT-2 Small reads a prompt of tokens and produces, at the last position, a probability for every next token. The **residual stream** is the running sum to which every attention and MLP block adds its output; $x_p \in \mathbb{R}^{768}$ is its value at position p at the hook point `blocks.L.hook_resid_pre` (layer $L = 8$ unless stated). For a prompt with true answer t, the **rank** of t is its place in the next-token ordering (1 is first), and the **blocker** b is the token ranked first. A prompt is *eligible* when t is not already first. **Success** means t becomes first after the edit. This is a low bar (Section 4.11); it is read together with side-effect measures and controls.

3.2 SAE Features and Delta Patching

For the activation x_p the SAE gives sparse feature activations and a reconstruction,

$$f_p = \mathrm{ReLU}\big(W_{\mathrm{enc}}(x_p - b_{\mathrm{dec}}) + b_{\mathrm{enc}}\big) \in \mathbb{R}^{24576}, \qquad \hat{x}_p = W_{\mathrm{dec}}^{\top} f_p + b_{\mathrm{dec}}, \tag{3.1}$$

where row $W_{\mathrm{dec}}[k]$ is the direction of feature k in the residual stream. Replacing x_p by $\hat{x}_p$ would inject the SAE's reconstruction error (12 to 26% of the activation norm) and disturb the model even with no edit. The system therefore adds only the change of the decoded output. If feature

k is scaled by s_k (with $s_k = 1$ for features left alone),

$$\Delta_p = \sum_k (s_k - 1)\, f_{p,k}\, W_{\text{dec}}[k], \qquad x'_p = x_p + \Delta_p, \tag{3.2}$$

at every prompt position p. Muting with strength $\mu \in [0,1]$ uses $s_k = 1 - \mu$ ($\mu = 1$ removes the feature); boosting with strength $\beta \geq 0$ uses $s_k = 1 + \beta$. With all $s_k = 1$ the edit is exactly zero. The hook (`make_scale_map_hook`) is removed after the pass, so the edit never outlives it and no weight changes.

3.3 Choosing Candidates: Causal Ranking

Ranking all 24,576 features by ablation would need as many forward passes. **Stage 1** screens: one clean pass caches the residual stream, the SAE encodes it, and the top N features by activation are kept, either at the last token or over *all* prompt positions except the start token, each scored by its maximum activation. **Stage 2** ranks causally: each candidate is removed alone and the change in a token's probability is recorded,

$$\Delta p_k(v) = P_{\text{ablate}(k)}(v) - P_{\text{clean}}(v), \tag{3.3}$$

for the blocker ($v = b$; the most negative values are its strongest drivers, the *mute list*) and for the target ($v = t$; the most negative values are its strongest helpers, the *boost list*). The N ablations are stacked into the batch dimension of one model call, so ranking costs two batched calls instead of $2N + 1$.

3.4 The Hybrid Mute and Boost Sweep

The **safety filter** tests every candidate alone at its strength and keeps it only if the target does not get worse:

$$\text{safe}(k) \iff r_k(t) \leq r_0(t) \ \wedge \ p_k(t) - p_0(t) \geq -10^{-6}, \tag{3.4}$$

with r and p the target's rank and probability with feature k applied alone (r_0, p_0: baseline). A feature safe on both sides is kept on one side only. The sweep then applies the first k mutes and first k boosts together in one forward pass, for $k = 1, 2, \ldots$, and stops when the target is first. The best step, not the last, is reported, since adding individually safe features is not monotonic. When both pools are used up, **pool refill** keeps every applied feature, makes one pass to obtain the *steered baseline* and its new blocker, and re-ranks and re-filters the features not yet applied (rejected ones included).

3.5 The Gradient-Descent Editor

The sweep decides which features and how many, with one strength per side. The gradient-descent editor gives each of $K = 200$ candidate features its own multiplier and tunes all of them for the given prompt. With free numbers z_k (initialised so that no feature is changed) the multiplier is $m_k = 1 + a_k$ with

$$a_k = -1 + 3\,\sigma(z_k), \qquad m_k \in (0,3), \tag{3.5}$$

where σ is the logistic function, and with $A_{p,k}$ the clean activation of feature k at position p the edit is Equation 3.2,

$$\Delta_p = \sum_{k=1}^{K} a_k A_{p,k} W_{\text{dec}}[k]. \tag{3.6}$$

Blocks 8 to 11 are run from the edited state and the logits ℓ at the last position are read. The loss is

$$\mathscr{L} = \max\bigl(0,\ \gamma - (\ell_t - \max_{v \neq t} \ell_v)\bigr) + \lambda_{\text{KL}}\,\mathbf{1}[\text{target leads}]\,\text{KL} + \lambda_{\text{size}} \sum_k |a_k|\, w_k, \tag{3.7}$$

with margin $\gamma = 0.3$, $\lambda_{\text{KL}} = 3$, $\lambda_{\text{size}} = 0.005$, and w_k the feature's share of the total maximum activation. The first term asks the target to beat every other token by γ logits; the second penalises a change of the rest of the distribution,

$$\text{KL} = \sum_{v \notin \{t,b\}} q_v \log \frac{q_v}{\tilde{p}_v}, \qquad q = \text{softmax}\bigl(\ell^{\text{clean}}_{\setminus\{t,b\}}\bigr),\ \ \tilde{p} = \text{softmax}\bigl(\ell_{\setminus\{t,b\}}\bigr), \tag{3.8}$$

over all tokens except the target and the blocker, so that the intended change is not penalised; it is charged only once the target leads, which mirrors the sweep's stopping rule; the third prefers small edits. Only the z_k are updated, by Adam [33] (learning rate 0.1, 100 steps); GPT-2's weights and the SAE are frozen and get no gradient. The best iterate is kept and re-checked through the same hook as the sweep in a full forward pass, so the reported rank does not depend on the tuning shortcut. Figure 3.1 shows the flow.

3.6 The Additive Edit

A multiplier cannot switch on a silent feature: $0 \times m = 0$ (Figure 3.2). The additive extension adds an amount $c_k \geq 0$ of the decoder direction of selected silent features, at the last position only:

$$\text{new volume} = \text{old volume} \times m + c, \qquad \Delta_{\text{last}} \mathrel{+}= \sum_{k \in \mathscr{S}} c_k W_{\text{dec}}[k]. \tag{3.9}$$

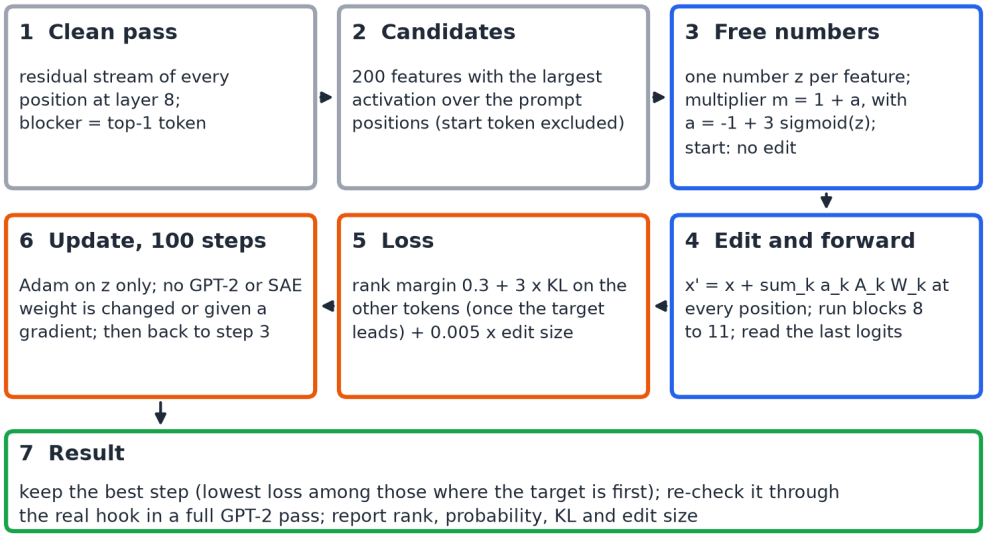


Figure 3.1: The gradient-descent editor. Only the free numbers z_k are updated.

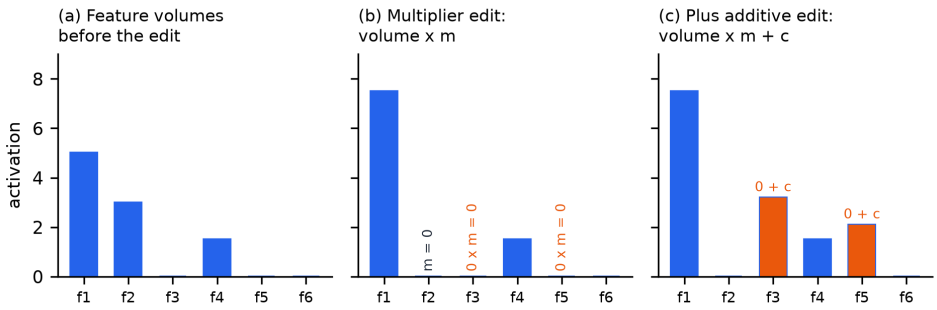


Figure 3.2: Why an additive edit was tried. A multiplier leaves a silent feature (f3, f5) at zero; an additive amount c switches it on.

The set $\mathscr{S}$ holds the 200 features that are exactly silent at the last position and have the largest positive first-order score $W_{\text{dec}}[k] \cdot g$, where $g = \partial(\ell_t - \max_{v \neq t} \ell_v)/\partial x_{\text{last}}$ is the gradient of the target's margin with respect to the residual. Each amount is $c_k = \text{cap} \cdot (\sigma(s_k) - \sigma(s_0))$ for $s_k \geq s_0$ (else 0), with every amount starting at exactly zero, cap the loudest feature on the real prompt positions, and an extra loss $0.005 \sum_k c_k/\text{cap}$. The score is the first-order Taylor estimate of the margin's change, which is why the edit writes the *answer's output direction* and not a representation of a fact (Section 4.11).

3.7 Benchmark and Metrics

The benchmark (16,360 CounterFact prompts, a subject-safe split, 300 test prompts) is described in Section 4.8. Metrics are the *rank* and *probability* of the target; *same-prompt KL* (Equation 3.8); *edit size*, the norm of the change over the norm of the residual at the last position; the effect of the same edit on 20 unrelated prompts, on CounterFact's reworded and

neighbouring prompts, and on generated text. Methods are compared on the same prompts with the exact McNemar test [35] on reaching rank 1 and the Wilcoxon test [36] on KL; choices were made on validation prompts and test prompts were only read.

Chapter 4

Evolution of the System: a Record of Trial and Error

This chapter tells how the system of Chapter 3 was reached. Each section states what we tried, what happened, and why we moved on, in the order of the research journal (Appendix A maps entries to sections). The early experiments used one prompt or 22 facts, so they explain why the design is as it is and are not estimates of performance.

4.1 The Starting Point

The first prototype (July 2026, journal 1) loaded GPT-2 Small and the layer-8 SAE and added a *causal selector*: take the features active on the prompt's last token, remove each in turn, and rank them by the change in the target's probability. Three findings shaped what followed. The most active feature is not the one that changes the answer, which is why we rank by causal effect and not by activation. Removing a feature completely damaged other predictions whereas a partial change (strength 0.2 to 0.3) did not on the one example tried. And an *eligibility rule* on 45 fact pairs from ROME's `known_1000` found 15 already correct, 22 *suppressed* (target not first but probable) and 8 improbable. The roadmap written then already named a learned predictor of which features to edit (reached in Section 4.9) and a comparison with ROME (still open).

4.2 Single-Feature Ablation

The first search ablated, one after another, the features that *help* the target (strength 0.3, up to five rounds) on the 22 suppressed facts (journal 2). It corrected **1 of 22**: muting the target's helpers weakens the target. The second version muted the drivers of the current *competitor*, the token ranked first, and corrected **3 of 22**, each in one round and with no change to 15 control prompts (journal 3). But all three started at rank 2, and it failed wherever the leader was a

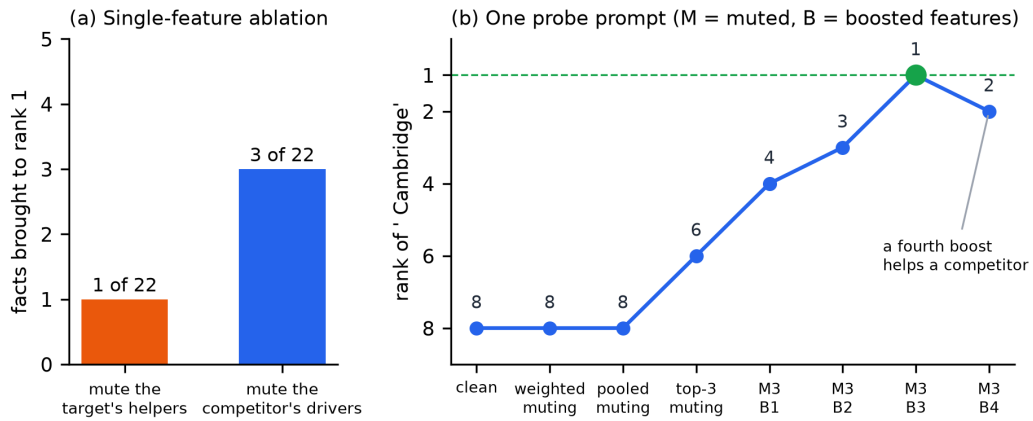


Figure 4.1: The early experiments. (a) 22 suppressed facts: muting the target's own helpers corrected 1, muting the competitor's drivers 3. (b) The probe prompt: muting alone stayed at rank 8; mute and boost reached rank 1 and a fourth boost undid it.

generic word such as " the" (Figure 4.1a). One feature was not enough; the next question was why.

4.3 Several Competitors, Spread-Out Support

On a probe prompt ("The location of Massachusetts Institute of Technology is in" → " Cambridge", rank 8, seven words above it) we muted all competitors in proportion to their probability (journals 4 to 8). The seven collapsed onto two features, 313 (" the", " a", " an") and 21169 (" question", " jeopardy", " danger", " doubt"), and the target stayed at rank 8. Raising the strength on feature 313 by 73% moved the target probability only from 1.29% to 1.39%, so under-muting was not the cause. Over each competitor's top ten features, influence falls off slowly for the semantic competitors (the second feature is 92 to 95% as influential as the first): the support is spread out. Muting the top three per competitor reached rank 6, five diluted it to 7. Muting alone was running out of road.

4.4 Mute and Boost Together

Adding the opposite move, boosting the features that support the target, changed the picture at once: three mutes and three boosts brought the probe prompt to **rank 1** (journal 5; Figure 4.1b). A fourth boost made it fall back to rank 2. This was not a bug (an isolated rerun gave the same 6.7357%): the fourth feature supports " the" more than " Cambridge", and because probabilities pass through a softmax, more edit is not always better. Two lessons were carried forward: report the *best* step, not the last, and check each feature against the target before applying it. The hybrid edit with a safety filter (rank 2 on this prompt, safer but weaker) became the working algorithm.

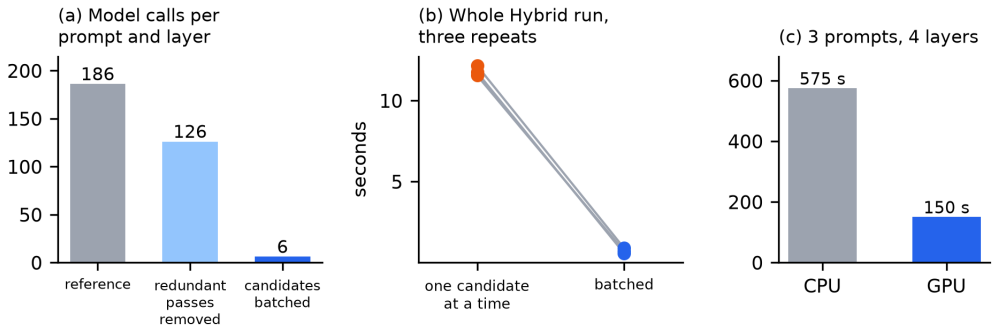


Figure 4.2: Making the computation fast. (a) Model calls per prompt and layer. (b) Whole Hybrid run, same result, three repeats. (c) CPU against GPU on 3 prompts and 4 layers.

4.5 Where to Edit, and Making It Fast

A layer sweep on 21 prompts (12 of them eligible) gave a plateau at layers 6 to 9, with layer 8 highest (mean rank gain 7.33, against 7.08, 6.50 and 6.33 at layers 7, 6 and 9 and at most 4.75 elsewhere; journals 10 and 11). Layer 8 was kept as a reasonable default, not a proven optimum.

A cost audit was the first real engineering step. One prompt at one layer needed **186 model calls**; both safety filters recomputed the clean baseline inside every candidate loop, and passing it in removed 60 calls with identical results. Stacking the candidates into the batch dimension of one call then cut the number to **6**, a 21-fold reduction, with identical ordered feature lists, probabilities (to 10^{-9}) and all 60 safety decisions; the sequential code was kept as the reference. A CUDA GPU gave a further 3.8 times, and in the application the whole Hybrid run fell from about 12 s to under 1 s (Figure 4.2).

4.6 Making the Sweep Reliable

With the sweep fast, its weaknesses showed (journals 16 to 19). It reported the last step instead of the best (fixed: best-so-far tracking); it added features in coarse batches (now one pair at a time); mute and boost as two chained hooks gave rank 66 where the combined safety check gave 59 (now one combined hook); and it stopped early because its pool held only features that passed the filter against the *clean* model. **Pool refill** fixed that by re-ranking against the edited model, and exposed a hard limit: for "The leading cause of death in men is" only 67 features were active at the last token, scaling cannot activate a feature that is off, and the run used all 67 and stopped at rank 51 (from 152).

The fix was to draw candidates from **all prompt positions**, since the meaning of "Sophia" sits on the word *Sophia*, not on the last word. On 13 comparable prompts all-positions reached rank 1 on **10** against 4 for the last token, won 9 and lost none (sign test $p \approx 0.004$), and in all 9 prompts where last-token failed it had used every feature it could see (Figure 4.3; journal

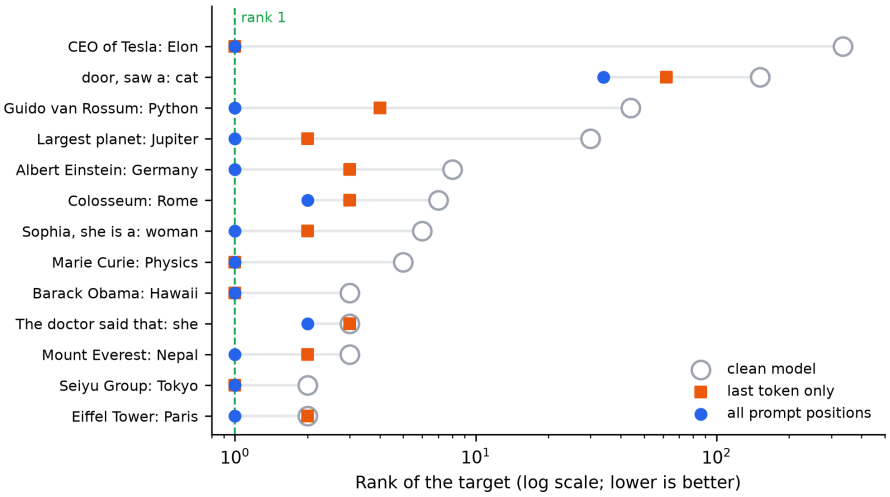


Figure 4.3: Final rank of the target on the 13 comparable prompts when candidates come from the last token only or from all prompt positions (log scale; rank 1 is the dashed line).

20). A filter study then asked whether the strict safety rule was too strict. In a 45-prompt pilot, removing the filter lost 10 of 29 successes and gained none, a 5% tolerance lost 2 and graded strengths changed nothing; on 131 last-token prompts relaxed rules added at most one prompt. Only 2.5 to 2.9% of features were rejected on both sides, so the supply of features, not the filter, was the binding limit and the strict filter stayed.

4.7 Is the Layer, or the SAE, the Limit?

Three worries were tested on the prompts that had failed in every mode (journal 22): that later layers rebuild what the edit removes; that the SAE is more machinery than a factual edit needs; and that the SAE's reconstruction error, which is never edited, hides the missing information. The edit's pushed direction was still 0.79 to 1.29 of its injected size at the output, and re-adding whatever fraction was lost gave the same rank in 21 of 21 layer and prompt cells, so **no repair was seen**. A free residual edit of the same size reached rank 1 in all 21 cells against 1 for the SAE edit, but the prompts were chosen because the SAE edit had failed on them, so this is a yardstick and not a success rate. The reconstruction error is 12 to 26% of the activation norm and untouched. A literature check that week (journal 25) found the field established and the evidence on SAE steering mixed, and led to the rule that every later result is compared with controls.

4.8 Scaling Up: the CounterFact Benchmark

The hand-assembled sets (22, 45, 131 prompts) share templates and are too small for held-out tests, so a benchmark was built from CounterFact (journal 24): of 21,919 records, 16,360 have

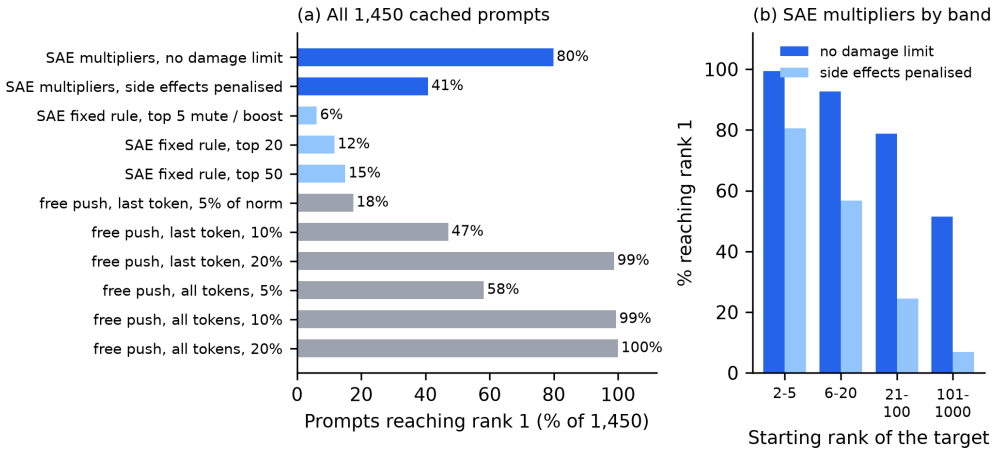


Figure 4.4: Headroom on the 1,450 cached prompts (one-shot edits on saved states, a stand-in for the real sweep): (a) share reaching rank 1 by method; (b) free multipliers by starting-rank band.

a single-token true answer that GPT-2 Small ranks 2 to 1000. A subject-safe split (a subject in one set; five relations held out) gave 1,000 training, 150 validation and 300 test prompts, and for 1,450 prompts the layer-8 states, candidate features, removal effects and the safety filter's verdict at ten strengths were cached once. A **headroom** measurement asked how much room any choice of multipliers has: free per-feature multipliers could bring the target to rank 1 on **80%** of the 1,450 prompts if side effects are ignored and on 41% with a side-effect penalty, against 6 to 15% for crude fixed rules (Figure 4.4).

4.9 Learning the Strengths

The sweep uses one mute strength (0.6) and one boost strength (0.5) for every prompt, although strengths change the outcome. A small network was trained to choose them (journals 23, 26, 27): *PromptNet* reads 12 summary numbers of the prompt (such as the target's rank and probability) and outputs a mute and a boost strength; *FeatureNet* outputs one multiplier per candidate. Training back-propagated through blocks 8 to 11 from the cached states. Against them stood a *learned fixed pair* (two numbers shared by all prompts) and a *per-prompt oracle* tuned by gradient descent at test time. On the 300 test prompts, under a stand-in measure, fixed 0.6/0.5 reached rank 1 on 75 (25.0%), the learned pair (mute 0.98, boost 1.81) on 116 (38.7%), PromptNet on 119 and the oracle on 123 (Figure 4.5). The gain came from a better *global* pair; choosing per prompt added 3 prompts (exact McNemar $p = 0.25$), side effects roughly doubled, and FeatureNet (31 to 32%) stayed below the pair. The real sweep was never run with the learned strengths. The network was not the useful part; its *teacher* was: free multipliers tuned on each prompt reached 73% in the same stand-in, nearly twice the network.

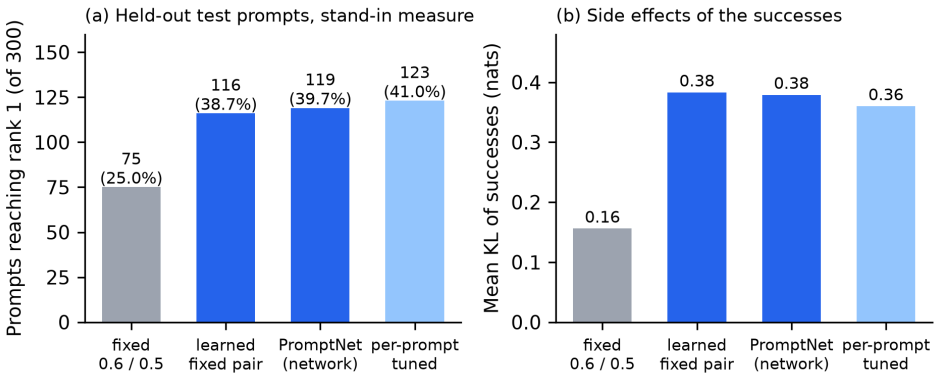


Figure 4.5: Learned strengths on the 300 test prompts (stand-in measure): (a) prompts reaching rank 1; (b) mean KL of the successes.

4.10 Gradient Descent Enters

Gradient descent had been in the project for a week before it became the method. It first appeared as a measuring stick: the bound for an edit made directly in the residual stream (journal 22, Adam on a free vector), then the headroom measurement (journal 24) and the oracle (journal 27). Its result, 73% against 31 to 32% for the network, suggested using the optimiser itself as the editor (journal 28, October 3): one multiplier per candidate, tuned for the prompt by 100 Adam steps, about 6 to 8 s per prompt. Its lineage in the literature is in Section 2.4: ROME, soft prompts and optimised steering vectors all find an edit by gradient descent on the target's probability.

The first hand trials favoured it but were unfair (it could scale features 0 to 3 times, the sweep 0.4 to 1.8), so the controlled comparison (journal 29) gave the sweep the same range (mute 1.0, boost 2.0) and measured all arms with one yardstick through the real hook. On the 300 test prompts gradient descent brought the true answer to rank 1 on **271** (90%), the wide sweep on 161 and the original sweep on 94 (Figure 4.6). It solved 110 prompts the wide sweep missed and lost none, its same-prompt KL was lower on 160 of the 161 prompts both solved, and it needed 7.7 s per prompt against 38.5 and 49.7 s. Two cautions stand: part of the KL advantage is built in (its loss contains a KL term and the sweep has none), and the median target probability at rank 1 is only 7.7%. It was weakest in the hardest band (54 of 79 prompts starting at rank 101 to 1000).

4.11 Forcing Every Target to Rank 1: the Additive Edit

The 29 prompts gradient descent failed on are mostly deep targets whose helpful features may be silent, and a multiplier cannot switch on a silent feature. The additive edit of Section 3.6 could, and its first hand trials were dazzling: all six hard prompts tried, including absurd targets such as a random name after "Hello, my name is", reached rank 1. That was the warning. A

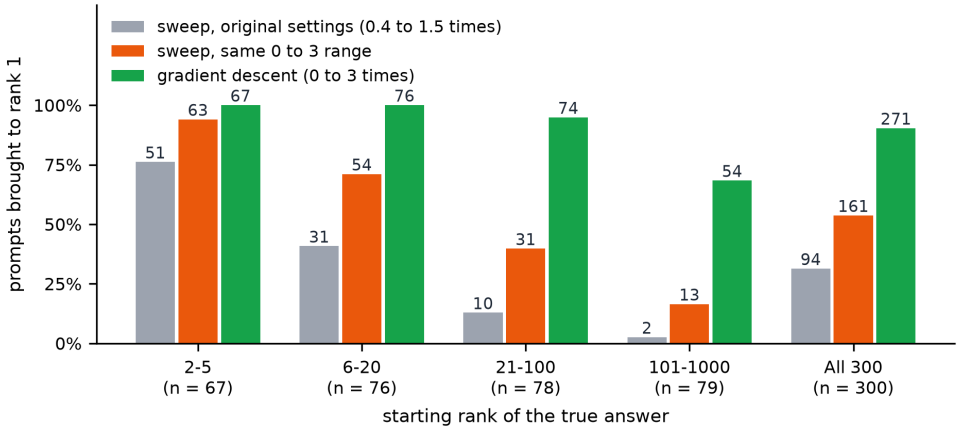


Figure 4.6: Prompts brought to rank 1 by starting rank of the true answer, 300 held-out prompts: the original sweep, the sweep with the same 0 to 3 range, and gradient descent (green).

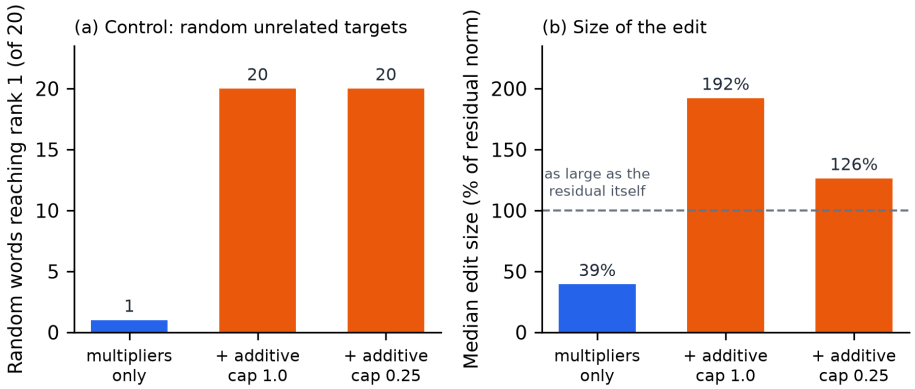


Figure 4.7: Control with random unrelated targets: the additive edit reaches rank 1 on 20 of 20 prompts at a size above the residual stream's; multipliers on 1 of 20.

control paired 20 validation prompts with random unrelated words: multipliers reached rank 1 on 1 of 20, the additive edit on **20 of 20**, with an edit 126 to 192% of the size of the residual stream itself (Figure 4.7). It is the analogue of fitting random labels [48] or of an adversarial perturbation [46, 47]: given enough freedom an optimiser reaches any chosen output. Its transfer to reworded prompts was as large for a random word as for the true answer (median rank 4 against 4.5; rank 1 on 25% against 16%), and the same edit changed the top word of 82% of unrelated prompts. The edit *steers toward a word*; it does not repair a fact.

Three consequences followed. Rank 1 is a weak test, so every later result needs controls. Reachability by steering is no evidence about knowledge in either direction, so the early label "knowledge absent" was withdrawn as a statement about the model. And the additive edit was switched off by default, with a warning. It answered "can we force GPT-2 to say what we want?" with "almost anything", which is exactly why it is not a fix.

Table 4.1: What the trial-and-error record kept and what it dropped.

Kept	**Dropped, and why**
Delta patching: reconstruction error never injected	Muting the target's helpers: weakens the target
Causal ranking by ablation, in batches	Weighted, summed or top-K muting alone: support is spread
Mute and boost with a strict per-feature filter	Relaxed or graded filters: at most one prompt gained
Best-step reporting, pool refill, candidates from all positions	Last-token candidates: features run out
Gradient-descent editor (271 of 300)	Learned strength network: no gain per prompt
Controls, held-out benchmark, fixed pass bars	Additive edit as a fix, and the "knowledge absent" label

4.12 What the Edit Does Beyond the Next Word

The multiplier edit was then tested on 40 test prompts (10 per band) for what it does besides rank 1 (journal 30). *Reworded prompts:* tuned on one prompt and applied unchanged, it lowered the true answer's median rank from 94 to 36 and reached rank 1 on 9% of rewordings, whereas a random word fell from 17,504 to 11,832 and reached it on none; the fact-related part is modest and the edit is mostly local. *Unrelated prompts:* the multiplier edit changed the top word of 6% of 20 prompts, the additive edit 17 to 24%. *Generated text* (20 greedy tokens, edit kept on): fluency unchanged (perplexity 4.4 to 4.8 against 4.7), variety slightly lower (distinct 2-grams 0.85 to 0.89 against 0.93), loops in 2 to 12% of continuations against none, the target repeated about 0.6 to 0.7 extra times. *Instructions:* with the prefix "Do not say the word X", the unedited model already put X first in 45% of prompts (a named word primes copying), the multiplier edit raised this to 68% and the additive edit to 88 to 90%: the edit ignores the instruction.

4.13 Where the Main Line Stands

Table 4.1 summarises the record. A transient SAE edit brings a suppressed answer to the top on most held-out prompts, cheaply and with small side effects on the same prompt, but it does so by pushing toward a word, it carries little beyond the prompt it was tuned on, and it ignores instructions. Three open limits remained: an edit is tied to one model that has an SAE; GPT-2 Small often lacks answers that larger models hold; and rank 1 needs controls to mean anything. Together they suggested a question for the tools we had built: could the change an edit makes in one model be carried into another? Chapter 5 reports that applied study.

Chapter 5

Applied Study: Carrying an Edit to Another Model

This chapter is a *side study*. It is not the next version of the system of Chapters 3 and 4; it applies the tools that work produced (the edit, the benchmark, the controls and the habit of fixing pass bars first) to a new question, to see whether the interpretability and intervention work has fruits beyond one model. It was done in October 2026 (journal 31 to 37).

5.1 The Question and the Design

Can the change an edit makes inside GPT-2 Small be translated and added to GPT-2 Medium at inference time, with no weight change, and can Medium's state improve Small? We call the first direction **Export** (small to medium) and the second **Import** (medium to small); Figure 5.1 shows both. The two models do not share internal coordinates (width 768 against 1024), so a *translator* is needed, and Section 4.11 showed that "the answer reaches rank 1" is weak evidence, so every test carries controls.

Models and data. Small is GPT-2 Small with its layer-8 SAE; medium is GPT-2 Medium (24 layers, width 1024, no SAE); both share one tokenizer. Small's layer 8 is paired with medium's layer 16. The benchmark is 200 CounterFact records from the hard set of Section 4.8, split into 50 development (dev) and 150 test records; dev records choose settings (layer, dose, epoch) and test records are only read. The edit tuned in small reaches rank 1 on 116 of the 200; the **88 primary test records** are those test records on which it does, since only there is there an edit to carry. Medium's median rank of the target before anything is done is 142.

The translator. Only the *change* of an edit is translated, so the bias of an affine map cancels and the translated change is $\Delta_S W$ (zero at the first position). Four ways of obtaining W were tried: (1) a *ridge map* fitted on ordinary text, $W = \arg\min \|(H_S - \mu_S)W - (H_M - \mu_M)\|^2 + \lambda \|W\|^2$, over paired states of the two models on wikitext-2 (500,126 fit tokens) [41];

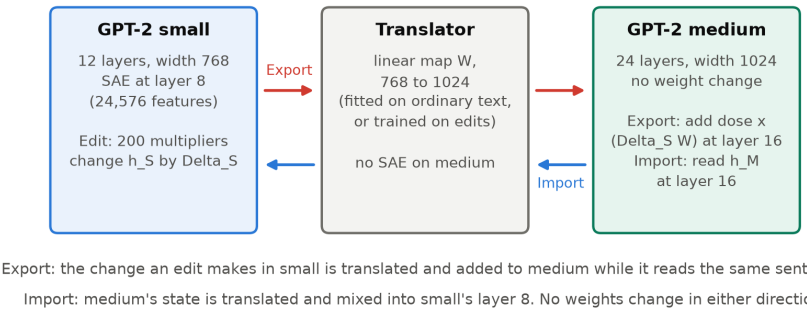


Figure 5.1: The two directions studied. A translator maps the residual stream of one model into the other's coordinates; no weight changes, and the SAE is used only where the edit is made.

(2) a *neural map* (ridge plus a one-hidden-layer correction); (3) a *linear map trained on edits*, started from the ridge map and trained so that medium's answers move as small's do, with goal $q = \text{softmax}(\ell_M + \ell_S^{\text{edit}} - \ell_S)$ and loss $\text{KL}(q \,\|\, p_M(h_M + \alpha \Delta_S W))$ over 289 successful training edits; and (4) *no fixed map*: the 200 multipliers re-tuned for each sentence against *medium's* output, a ceiling and not a transfer, since it looks at medium every time.

Export, Import and controls. Export adds the translated change while medium reads a sentence, $h'_M = h_M + \alpha \Delta_S W$ at layer 16 at every position but the first; the **dose** α is chosen on dev (from step D10 on by the best mean log-rank gain, since dev top-1 on 28 records proved fragile). Import mixes small's state with medium's translated state, $h'_S = (1-\gamma)h_S + \gamma T(h_M)$, with $\gamma = 1$ full replacement. The receiver is never shown the answer. Controls: a random vector of the same size; another record's edit; an edit toward a random word of matched starting rank; a *word push* along the target's output direction (it uses the answer, so it is a comparison and not a method); the fact stated in medium's prompt; for Import, a state from a wrong sentence, the average state and a random map. *Rewordings* and *neighbours* are the shares of reworded prompts on which the new answer beats the true one, and of other-subject prompts on which the true answer still wins. **Every pass bar ("gate") was written into the project plan before the run that judged it and never moved**; a step designed after seeing a result is labelled post-hoc and cannot change a verdict (Appendix B).

5.2 The Channel Works

The ridge map fits well enough to keep the receiver working: per-dimension R^2 is 0.58 (small layer 8 to medium layer 16) and 0.66 (medium 16 to small 8). Stitching one model's first layers to the other's last layers through the map recovers 91% (Export direction) and 98% (Import direction) of the receiver's own quality. The map copies *which word changed* well (cosine 0.63 to 0.76) but the summary at the last position only weakly (0.24 to 0.31), so the translator passes its pre-set bar for Import and not for Export.

Table 5.1: Export of the edit through the ridge map (88 primary test records; dose chosen on dev).

Arm	Top-1	Median rank	Rewordings	Neighbours
Medium unchanged	0%	142	36%	67%
Translated edit	**8%**	**14**	**52%**	**63%**
Random vector	0%	157	38%	66%
Another record's edit	0%	128	36%	65%
Word push (uses the answer)	83%	1	82%	23%
Fact stated in medium's prompt	–	–	97%	44%

A positive control swaps one country for another in a template sentence: the difference of the source model's states is translated and added to the receiver, and a hit is counted when the receiver's top answer becomes the swapped capital. The translated change succeeds on **55%** (Export) and **33%** (Import) of test cases against at most 0.4% for a random vector, a random map and another pair's difference. It is a channel check, not an edit or a fact.

5.3 Export: a Real but Weak Effect

For each of the 88 primary test records the edit tuned in small is relayed to medium through the ridge map and added at layer 16 (Table 5.1). The effect is real, graded and specific: the target's median rank in medium falls from 142 to **14**, it becomes the top answer on **8%** of records, reworded prompts favour the new answer on 52% against 36% unchanged, and neighbours lose only 4 points. A plain word push makes the target first on 83% of records but only 23% of neighbours keep the true answer, and stating the fact in medium's prompt costs 23 points of neighbours. The pre-set bar (top-1 of at least 15%, 10 points above the strongest control) was not met.

5.4 Searching for a Better Translator

Because the effect is real but weak, we tried seven changes, each with its pass rule written first (Figure 5.2; Appendix B). **Layer and capacity:** injecting at layers 4, 8, 12, 16 and 20 gave 0, 5, 5, 8 and 1%, and splitting the dose over several layers did no better, so the layer is not the limit; a neural map fits ordinary text much better (R^2 +0.09 to +0.11) but exports no better (8%), so a better fit to text does not help. **A map trained on edits** (method 3) roughly doubled the result to 18% top-1 (median rank 14 to 7, paired $p = 5 \times 10^{-9}$) and stayed specific to the record, but more training edits, longer training and a direct rank-1 term in the loss left it at about 20%. (One dose rule, dev top-1 on 28 records, gave a misleading 2% against 20% and was replaced, post hoc, by the dev log-rank gain.)

Why it plateaus. With the rank-1 term the map learned its 289 training targets (95% top-1

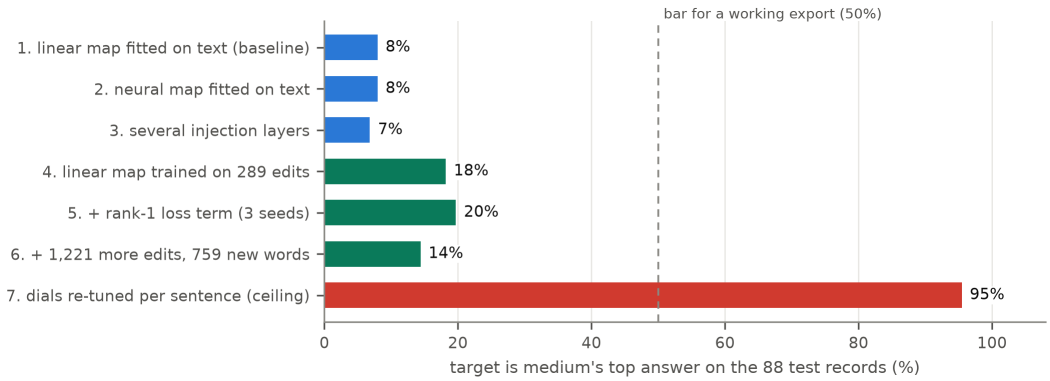


Figure 5.2: Top-1 of every export variant on the same 88 test records. Blue: map fitted on text; green: map trained on edits; red: per-sentence tuning against medium's output (a ceiling, not a transfer).

on them) but not unseen words (16%): it memorises. Giving it 1,221 more edits toward 759 random target words removed the memorisation but unseen records got *worse*, 20% to 14%, because the loss asks medium to copy small's shift, not to reach rank 1. **Per-sentence tuning is a ceiling:** re-tuning the multipliers against medium's output reaches the top answer on 95% of records (84% on all 150), with the same neighbour cost as the source edit inside small (8 points against 9). The channel can carry a rank-1 edit; what fails is the compatibility of an edit tuned for small with medium. The ceiling's solution is only partly in the direction of the original edit (cosine 0.51 against 0.02 by chance), so no map that merely transforms the given edit can supply what is missing; it also reaches 83% for a random word.

5.5 What Is Carried: a Fact or a Push?

The same question as in Section 4.11 applies to Export. Each arm's gain on reworded prompts must be measured against its *own* unchanged level (a random word starts lower, at 25% against 36%). Doing so, the per-sentence multipliers gain 24.4 points for the real target and 17.0 for a random word (difference +7.4, 95% interval 0.0 to +15.3, $p = 0.031$), and the map trained on edits gains 19.9 and 18.2 ($p = 0.36$). Neither difference passes the $p < 0.01$ line fixed beforehand. An earlier "four times" claim compared the wrong baselines and was withdrawn. What carries to reworded prompts is a push toward the targeted word, not something specific to the fact. A further control repeated the per-sentence tuning with 200 random directions in place of the 200 SAE directions: the SAE directions reached 95% and the random ones 81%, a difference of 15 points, so the SAE makes the tuning more effective but is not what makes steering possible.

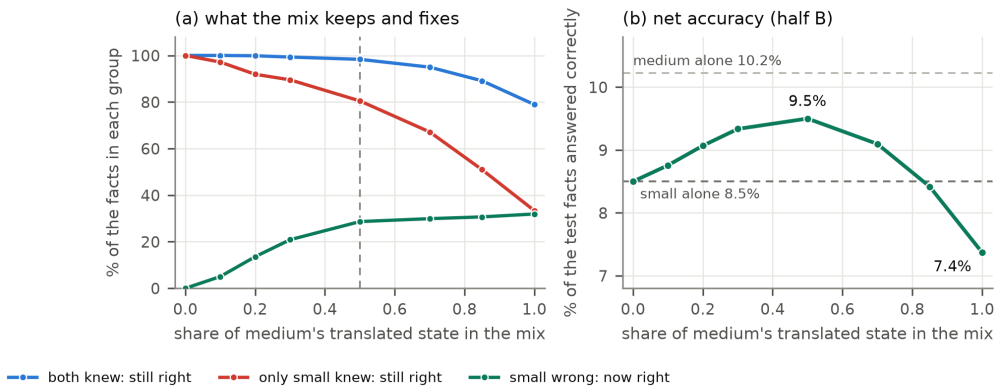


Figure 5.3: Import by mixing small's own state with medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy: best at a 50/50 mix, below small alone under full replacement.

5.6 Import: Medium's State into Small

Of the 21,789 single-token CounterFact facts left after the development and pilot records are removed, medium alone gets 9.9% right and small 8.3%; 780 facts are right in medium and wrong in small. **Full replacement** of small's layer-8 state by medium's translated state rescues 262 of the 780 (33.6%; controls at most 0.8%), but breaks 22% of the facts both models knew and 70% of those only small knew, so small's accuracy falls from 8.30% to 7.17%. The pre-set bar (50% rescue, accuracy half-way to medium, 90% of known facts kept) was not met, although the controls show that real information crosses.

A **softer mix** was then tried, with its rule written before the build: $\gamma \in \{0.1, \ldots, 1\}$ chosen on one half of the facts and reported on the other (10,895 facts). Mixing in half of medium's state ($\gamma = 0.5$) raises small's accuracy from 8.5% to **9.5%** (medium alone: 10.2%), gaining 162 facts and losing 53 ($p = 2.5 \times 10^{-14}$), keeps 98% of the facts both models knew and rescues 28.6% of the facts small gets wrong (Figure 5.3). This meets the rule (gain of at least 0.5 points, 90% of known facts kept, $p < 0.01$) and is the first net-positive Import result. It is a one-point gain, it needs medium to run on the same sentence, and the replacement test's controls were not rerun for the mix.

5.7 What the Side Study Shows

A linear channel between GPT-2 Small and Medium exists and keeps the receiver working. The change an edit makes in small moves medium's answers specifically, in a graded way and without damaging its neighbours, but every translator that does not look at medium's output stops at about 20% top-1, far below the 50% bar, while looking at medium for every sentence reaches 95%. Together with the controls this puts the limit on the *edit's compatibility*, not on the channel, the layer, the translator's capacity or the amount of data, and what crosses is a push toward the targeted word and not a fact. Limits: one model pair, one dataset and one

SAE; 88 primary test records; the additive edit, generated text and other pairs untested; and the map follows earlier stitching work [40].

Chapter 6

Implementation

6.1 Software, Hardware and Application

The system is written in Python with PyTorch [20], TransformerLens 3.5.1 [13] (named hook points on every GPT-2 activation, and running the model from a saved intermediate state), SAELens 6.45.3 [14] (loading the `gpt2-small-res-jb` dictionaries [15]), Neuronpedia [18] for feature descriptions and Streamlit for the application. The main-line studies ran on a GTX 1660 Ti PC (PyTorch 2.6.0, CUDA 12.4); caching, headroom, the strength models and the transfer study ran on an RTX 4050 laptop (PyTorch 2.5.1, CUDA 12.1), both with 6 GB of memory. The code also runs on Apple silicon and the CPU; the CPU path was verified. Timings are compared only on one machine.

The application `experiment_app.py` has seven tabs in three tiers: the interface, the intervention engine and the model and data layer (cached GPT-2, SAEs, fact sets, per-prompt caches). *Hybrid mute and boost* carries both editors behind a choice of editing method (the fixed sweep with its strengths, cumulative sweep, pool refill, safety filter and GPU-batched switch, or gradient descent with its steps, learning rate, margin and weights), with a live tracker, a stopwatch and a session-history record. *Prototype lab* runs gradient descent with the optional additive edit, shows baseline and edited text side by side and applies the same edit to follow-up prompts. *Batch* runs studies in background workers; *Repair & SAE limit* shows the diagnostics of Section 4.7; *Sequential vs Batched Proof* times the two code paths; *Monosemanticity Analysis* inspects features; *Session history* stores every run. The edit lives in `src/hooks.py` (Equation 3.2), `src/editing.py` (screening, causal ranking and the safety checks of Equation 3.4), `src/hybrid_runner.py` (the headless sweep, which reproduces the application exactly) and `src/gradient_editing.py` (Equations 3.5 to 3.9). The transfer study adds only new files (`src/transfer/`, tools `00` to `27` in `tools/transfer/`); older functions are imported and never edited, so earlier results cannot change.

6.2 Correctness Checks and Engineering Lessons

Every fast path was checked against a slower reference before use, and the checks are kept as scripts: batched against sequential ranking (identical feature lists, probabilities to 10^{-9}, 60 of 60 safety decisions); the cached-state path against the full model (logits identical on one machine, within 10^{-5} on the other); a random per-feature edit through the cache path against the real hook (same target rank); and the gradient-descent tuning path against the real hook in every recorded run. Built-in sanity checks stop a run on a mismatch; one caught a cache-builder error (a multiplier 1.0 passed where 0.0 was meant) before any data was written. Three problems could have invalidated results without any error message.

- **A thread race removed the start-of-text token.** Tokenising with the start token switched off temporarily changes a setting shared by the whole model in TransformerLens; when two Streamlit sessions overlapped, the wrong value could be written back permanently and every later prompt was tokenised without its start token (rank 28 instead of 22 on one prompt). Of 57 recorded application runs, 16 had wrong baselines, all from one server process, and none was used in a table. The fix tokenises directly and never touches the model's configuration.
- **A timer measured the network** (46 s against about 10 s of model time), and **chained hooks disagreed with the combined check** (rank 66 against 59); look-ups moved out of the timer and one combined hook is used.
- **GPU memory.** A padded training set of 7,550 texts (3.8 GB) nearly filled the 6 GB card and made runs five to ten times slower without any error (6.2 hours instead of one). Training tensors now stay on the CPU.

6.3 Reproducibility and Development Process

Every tool prints its pass rule, saves its raw output as a log and writes a result file with per-run details. Logs and results of every experiment are copied into a data pack under version control, and the figures of this report are drawn from the pack by scripts that run no model; larger local artefacts (2.6 GB) are archived with a manifest of sizes and checksums. Optimisation followed a *one-bounded-step* rule: identify one bottleneck, measure the reference, make one change, verify equivalence on a fixed case, log, and only then approve the next step.

Chapter 7

Discussion, Conclusion and Future Work

7.1 What the Work Shows

The supply of features, not the selection rule, limited the sweep. Candidates from all prompt positions enlarged the pool about fivefold and raised the prompts at rank 1 from 4 to 10 of 13; removing the safety filter lost prompts while relaxing it gained at most one. **The choice of features and strengths mattered more than stronger interventions.** Muting the competitor's drivers and adding boosts gave the early gains; one better global strength pair lifted a stand-in success rate from 25% to 39%, choosing strengths per prompt added nothing measurable, and tuning one multiplier per feature on the prompt itself reached 271 of 300 on the real model. **Rank 1 is a weak test.** The additive edit forced random words to rank 1 on 20 of 20 prompts, and an optimiser with enough freedom reaches any chosen output; reachability by steering is not evidence about what the model knows, in either direction. The edit is a push toward the answer's output direction: it carries little beyond the prompt it was tuned on and largely ignores an instruction not to say the word. **The side study** showed that a linear channel between GPT-2 Small and Medium keeps the receiver working, that an exported edit moves the target specifically but rarely to rank 1, that what crosses is a push and not a fact, and that mixing in half of Medium's state raises Small's accuracy by one point. Four habits earned their cost: controls first (the random-word control turned the best-looking result into a warning), fair comparisons (giving the sweep the same range as gradient descent cut the gap but did not close it), pass bars fixed before the run (so the side study's failures stand as failures), and corrections made in the open (the "knowledge absent" class, a "four times" random-word claim and an inflated last-position cosine are withdrawn in the journal where they were made).

7.2 Threats to Validity and What Is Not Shown

- One model family, one older SAE release and layer 8 in all later studies; the early studies use small, templated prompt sets (13 to 131 prompts) and, like CounterFact,

treat prompts as independent without correction for multiple comparisons.

- Success is rank 1 on the next token (median target probability 7.7% for gradient descent). Side effects were measured on the same prompt, 20 unrelated prompts and 40-prompt tests, with one seed, no confidence intervals and true targets only for the generation and instruction tests.
- The headroom and the learned strengths are stand-ins on cached states; the real sweep was never run with the learned strengths. The side study has 88 primary test records and settings chosen on 28 to 50 dev records.
- **No comparison with ROME, MEMIT, in-context editing, prompting or difference-of-means steering has been run**, so nothing here says how the approach ranks against them.

7.3 Conclusion

FeatureScalpel set out to repair suppressed factual recall in GPT-2 Small through the features of a sparse autoencoder, without changing a weight. Through attempts that mostly failed in instructive ways it arrived at a transient edit that brings the true answer to rank 1 on 271 of 300 held-out prompts, quickly and with small side effects on the same prompt. The applied side study found that the change an edit makes can be carried between models as a push, that importing a larger model's state gives a small gain, and that a fact does not cross. The results support the claim that a transient SAE-feature edit can raise a suppressed answer to the top, and that optimising a multiplier per feature does so more often, faster and with lower same-prompt side effects than a hand-built sweep. They do not support the claim that such edits recover knowledge, are specific to a fact, or outperform weight editing, in-context editing or prompting.

7.4 Future Work

1. **A specificity benchmark** that measures, for every method, whether the target reaches rank 1, transfers to rewordings, leaves unrelated prompts and generated text unchanged and respects an instruction (with random words of matched starting rank as controls),.
2. **Head-to-head comparison** with ROME, MEMIT, in-context editing and difference-of-means steering using existing implementations [30], timed on one machine; this is the test of the project's central claim.
3. **Strengths, transfer and scale:** the learned strengths in the real sweep; controls for the Import mix; other model pairs (for example Gemma 3 270M and 1B, which have SAEs on both sides) and larger models with pretrained SAEs [17].

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[2] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 17359–17372.

[3] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[4] A. Gupta, A. Rao, and G. Anumanchipalli, "Model editing at scale leads to gradual and catastrophic forgetting," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024; arXiv:2401.07453.

[5] N. Elhage, T. Hume, C. Olsson, *et al.*, "Toy models of superposition," *Transformer Circuits Thread*, 2022.

[6] T. Bricken, A. Templeton, J. Batson, *et al.*, "Towards monosemanticity: Decomposing language models with dictionary learning," *Transformer Circuits Thread*, 2023.

[7] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse autoencoders find highly interpretable features in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[8] A. Templeton, T. Conerly, *et al.*, "Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet," *Transformer Circuits Thread*, 2024.

[9] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid, "Steering language models with activation engineering," *arXiv preprint arXiv:2308.10248*, 2023.

[10] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner, "Steering Llama 2 via contrastive activation addition," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15504–15522.

[11] A. Zou, L. Phan, S. Chen, *et al.*, "Representation engineering: A top-down approach to AI transparency," *arXiv preprint arXiv:2310.01405*, 2023.

[12] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, "AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders," in *Proc. International Conference on Machine Learning (ICML)*, 2025; arXiv:2501.17148.

[13] N. Nanda and J. Bloom, "TransformerLens," GitHub repository, 2022. [Online]. Available: https://github.com/TransformerLensOrg/TransformerLens

[14] J. Bloom, C. Tigges, A. Duong, and D. Chanin, "SAELens," GitHub repository, 2024. [Online]. Available: https://github.com/decoderesearch/SAELens

[15] J. Bloom, "Open source sparse autoencoders for all residual stream layers of GPT2 small," *AI Alignment Forum*, 2024.

[16] J. Kramár, T. Lieberum, R. Shah, and N. Nanda, "AtP*: An efficient and scalable method for localizing LLM behaviour to components," *arXiv preprint arXiv:2403.00745*, 2024.

[17] T. Lieberum, S. Rajamanoharan, A. Conmy, *et al.*, "Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2," *arXiv preprint arXiv:2408.05147*, 2024.

[18] J. Lin and J. Bloom, "Neuronpedia: Interactive reference and tooling for analyzing neural networks," 2023. [Online]. Available: https://www.neuronpedia.org

[19] S. Heimersheim and N. Nanda, "How to use and interpret activation patching," *arXiv preprint arXiv:2404.15255*, 2024.

[20] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[21] D. Arad, A. Mueller, and Y. Belinkov, "SAEs are good for steering – if you select the right features," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025; arXiv:2505.20063.

[22] L. Smith, S. Rajamanoharan, A. Conmy, C. McDougall, T. Lieberum, J. Kramár, R. Shah, and N. Nanda, "Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update #2)," *LessWrong*, Mar. 2025. [Online]. Available: https://www.lesswrong.com/posts/4uXCAJNuPKtKBsi28/negative-results-for-saes-on-downstream-tasks

[23] M. Chaudhary and A. Geiger, "Evaluating open-source sparse autoencoders on disentangling factual knowledge in GPT-2 small," *arXiv preprint arXiv:2409.04478*, 2024.

[24] M. Cui, L. Shen, and X. Yang, "SAE interventions are unreliable: Post-intervention recovery of suppressed behavior," *arXiv preprint arXiv:2606.18322*, 2026.

[25] E. Farrell, Y.-T. Lau, and A. Conmy, "Applying sparse autoencoders to unlearn knowledge in language models," *arXiv preprint arXiv:2410.19278*, 2024.

[26] S. Chalnev, M. Siu, and A. Conmy, "Improving steering vectors by targeting sparse autoencoder features," *arXiv preprint arXiv:2411.02193*, 2024.

[27] Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, X. He, K.-F. Wong, and P. Minervini, "Steering knowledge selection behaviours in LLMs via SAE-based representation engineering," in *Proc. NAACL*, 2025; arXiv:2410.15999.

[28] P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun, "Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[29] C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang, "Can we edit factual knowledge by in-context learning?" in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023; arXiv:2305.12740.

[30] P. Wang, N. Zhang, B. Tian, *et al.*, "EasyEdit: An easy-to-use knowledge editing framework for large language models," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024; arXiv:2308.07269.

[31] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, "Sparse feature circuits: Discovering and editing interpretable causal graphs in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2025; arXiv:2403.19647.

[32] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[33] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[34] nostalgebraist, "Interpreting GPT: the logit lens," *LessWrong*, Aug. 2020. [Online]. Available: https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens

[35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[36] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[37] K. Lenc and A. Vedaldi, "Understanding image representations by measuring their equivariance and equivalence," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[38] Y. Bansal, P. Nakkiran, and B. Barak, "Revisiting model stitching to compare neural representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[39] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proc. International Conference on Machine Learning (ICML)*, 2019.

[40] A. Chen, J. Merullo, A. Stolfo, and E. Pavlick, "Transferring linear features across language models with model stitching," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025; arXiv:2506.06609.

[41] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[42] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[43] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[44] N. Subramani, N. Suresh, and M. E. Peters, "Extracting latent steering vectors from pretrained language models," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022, pp. 566–581.

[45] J. Dunefsky and A. Cohan, "One-shot optimized steering vectors mediate safety-relevant behaviors in LLMs," in *Conference on Language Modeling (COLM)*, 2025; arXiv:2502.18862.

[46] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2014.

[47] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[48] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

Appendices

Appendix A: Research Journal and Report Sections

The journal (`docs/Research_Journal/`, entries 1 to 37) records every step with its tables, rules, mistakes and corrections. Result files and raw logs of the later entries are in `docs/Research_Journal/packs/`.

Table A.1: Journal entries and where they appear in this report.

Entries	Content	Section
1	First prototype, causal selector, eligibility rule, roadmap	4.1
2, 3	Target-helper and competitor ablation (1 and 3 of 22)	4.2
4, 6, 7, 8	Weighted multi-competitor muting, aggregation, decay profile, top-K	4.3
5	Hybrid mute and boost, rank regression	4.4
9, 10, 11	SAE audit, layer benchmark, caching and profiling	4.5
12, 13, 14, 15	GPU, redundant-pass removal, batching (186 to 6 calls), batched application	4.5
16, 17, 18, 19	Best-so-far, cumulative sweep, combined hook, pool refill, all-position candidates	4.6
20, 21	Candidate-source study (10 of 13 against 4 of 13); safety-filter study	4.6
22	Repair and SAE-limit diagnostics	4.7
23, 24	Learned-strength plan; CounterFact hard set, split, cache, headroom	4.8
25	Literature check	4.7, Ch. 2
26, 27	Learned strengths (PromptNet, FeatureNet)	4.9
28, 29	Gradient-descent editor; equal-budget comparison (271, 161, 94 of 300)	4.10
30	Additive edit, random-word control, rewording, generation and instruction tests, start-token bug	4.11, 4.12
31 to 37	Cross-model transfer: translators, country swap, Export, search ladder, Import, SAE control	Ch. 5

Appendix B: Pass Bars of the Side Study and Their Verdicts

Every bar was written into the project plan, and committed, before the run that judged it; none was moved afterwards. "Edit's own" means the effect that the source edit has inside small itself. Verdicts are as printed by the tools.

Table B.1: Pass bars ("gates") and their verdicts.

Step	Bar fixed before the run	Result	Verdict
Gate 1 (A2)	Translator: difference cosine at the last position at least 0.3	Import pairs 0.31; Export pair 0.28	Import only
Gate 2 (B1)	Country swap beats the strongest control by a margin, paired test	55% and 33% against at most 0.4%	pass
Gate 3 (D)	Export top-1 at least 15% and 10 points above the strongest control	8%; margin 3 points	fail
Gate 4 (D5)	Best layer set at least 10 points above the best single layer, $p < 0.01$, top-1 at least 15%	7% against 8%; $p = 0.016$	fail
Gate 5 (D6)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 80% of the edit's own; neighbours fall $\leq$ 5 points	95%; 0.91; 8 points	fail (neighbours)
Gate 6 (D7)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 0.8; neighbours fall no more than the edit's own	18%; 0.74; 7 points	fail
Gate 7 (D10)	As Gate 6, plus real target 10 points above the random word	20%; 0.74; met; met	fail
Gate 8 (Import)	Rescue $\geq$ 50% of the 780 facts; accuracy halfway to medium; 10 points above controls ($p < 0.01$); 90% of known facts kept	33.6%; 7.2% against 9.1%; met; 78.3%	fail
Gate 9 (D13)	Gate 7 unchanged, on the full varied training set	14%; 0.62; met; met	fail
Rule E5	Soft Import: gain $\geq$ 0.5 points on held-out facts, 90% kept, $p < 0.01$	+1.0 point; 98.3%; $p = 2.5 \times 10^{-14}$	met
Rule S1	SAE basis "matters" if top-1 is higher by 10 points or rewordings lift by 5	+15 points; +2 points	matters
Rules S2, S3	Real target's rewordings lift larger than a random word's, $p < 0.01$	$p = 0.031$ (D6); $p = 0.36$ (D7)	not shown

Two notes written after the fact do not change any verdict: the 5-point neighbour line of Gate 5 was stricter than the source edit's own loss of 9 points inside small, so Gates 6, 7 and 9 use the edit's own loss; and the dose rule (dev top-1 on 28 records) was replaced by the dev log-rank gain from step D10 on, after step D9b showed it to be fragile.

Appendix C: Reproducing the Results

From the repository root, with `HF_HUB_OFFLINE=1` once the models are cached. The main line is on the branches `layer-intervention-benchmark`, `pool-refill-implementation` and `gradient-descent-editing`; the side study is on `cross-model-transfer`.

```
streamlit run experiment_app.py
python run_batch.py data/candidate_source_batch_spec.json
python tools/build_counterfact_set.py
python tools/split_counterfact_set.py
python tools/build_strength_cache.py
python tools/measure_headroom.py
python tools/train_strength_models.py
```

```
python tools/compare_sweep_vs_gradient.py --n-per-band 0
python tools/control_random_targets.py
python tools/transfer/00_setup_check.py   (then 01 to 27)
python tools/make_report_v3_history_figures.py
```

In order: the application; the candidate-source study; the CounterFact hard set and split; the per-prompt cache and the headroom measurement; the strength networks; the 271, 161 and 94 of 300 comparison; the random-word control; the side study; and the early-stage figures.

Journal entries 31 to 37 give each side-study tool with its run time, and `docs/cross_model_transfer/PLAN.md` holds all pass bars and readings.